TAPRECO

Technical Architecture, Hosting, Databases, APIs and GitHub Repositories

Prepared: 13 November 2026 **Scope:** architecture described in prior project discussions

Verification note: This document is a documented understanding, not a live audit of GitHub, Cloudflare, Render, Neon, or Supabase. Unknown counts and exact URLs are explicitly marked for confirmation.

1. Executive overview

Tapreco is a suite of US tax-related tools. Four custom React applications use a shared Node.js/Express tools API. A separate vendor Tax Planner is integrated into the user experience. A Roth Strategy Analyzer has been discussed as a planned tool. The architecture aims to offer a common login and consistent navigation across the suite.

2. Tool inventory

Tool	Function	Known state
Tax Planner	Vendor tax planning and scenarios	Vendor application / integrated
Mutual Fund	Federal and state mutual fund tax calculations	Custom frontend hosted
Estimated Tax	Estimated tax payment calculations	Custom frontend hosted
Audit Risk Analyzer	Form 1040 risk findings and PDF reports	Custom frontend hosted
Entity Comparison	Compare Schedule C and S-Corp outcomes	Custom frontend hosted
Roth Strategy Analyzer	Roth conversion strategy analysis	Planned; deployment unverified

Count: 6 named tools in the suite roadmap; 4 custom tool frontends discussed as hosted, 1 vendor application, and 1 planned tool. This does not mean all 6 are live.

3. Hosting and ownership by layer

Layer	Hosting / service	What it does
Domain / DNS	GoDaddy	Manages domain and DNS records; points hostnames to deployed services
Website / portal	Cloudflare Pages	Hosts static frontend build and portal UI
Four custom frontends	Cloudflare Pages	Serves compiled React/TypeScript/Vite assets
Shared tools backend	Render Web Service	Runs Node.js + Express APIs and server-side logic
Custom tools data	Supabase PostgreSQL	Stores application data; accessed by backend

Layer	Hosting / service	What it does
Vendor Tax Planner data	Neon PostgreSQL	Vendor-controlled database; do not modify vendor tables
Source control	GitHub	Branches, commits and deployment source

4. System architecture - request path

Typical custom-tool flow:

Step	Component	Interaction
1	User browser	Opens Tapreco portal or tool URL
2	Cloudflare Pages	Returns React HTML, JavaScript and CSS
3	React app	User enters data; frontend validates input and makes HTTPS API request
4	Render tools-api	Express route authenticates/authorizes request, validates data and executes calculations
5	Supabase PostgreSQL	Backend queries or writes data over TLS
6	Render tools-api	Returns JSON response, error details, or generated report payload
7	React app	Renders results and may offer PDF/Excel download

Logical diagram: Browser / React (Cloudflare Pages) -- HTTPS / JSON --> Node.js / Express (Render) -- PostgreSQL / TLS --> Supabase. The response travels back in reverse order.

Important: React should not connect directly to PostgreSQL using server credentials. Database passwords, private keys, and privileged queries belong on the backend.

5. Example React-to-API integration

Frontend concept: A React screen calls an API base URL configured in the Vite build environment (for example, VITE_API_BASE_URL). It sends JSON with fetch or an HTTP client.

```
POST /api/estimated-tax/calculate
Content-Type: application/json
{ "income": 120000, "filingStatus": "single" }
```

Backend concept: Express receives the request, validates the fields, checks authorization, performs calculations and, when needed, reads/writes PostgreSQL through a server-side connection pool. It returns structured JSON to React. The route above is **illustrative**, not a verified route in your repository.

Cross-origin configuration: If Cloudflare frontend and Render API use different origins, the API must allow only approved frontend origins through CORS. Authentication cookies require appropriate SameSite, Secure, domain and credential settings.

6. Databases and connectivity

Database	Known purpose	Important rule
Supabase PostgreSQL	Custom tools / tools-api application data	Backend-only access, SSL/TLS; prior pooler port 6543
Neon PostgreSQL	Vendor Tax Planner data	Do not alter vendor tables or schema without vendor approval

Previous Render deployment troubleshooting involved Supabase TLS certificate validation and a missing DATABASE_TLS_CA setting. Keep CA certificates and database connection strings in Render environment variables/secrets, not in frontend code or GitHub. The exact active database instances and schema names require verification.

7. GitHub repositories and folder structure

Confirmed from previous discussions: A Tapreco repository has been referenced at Upsilon-GDC/tapreco, with later mentions of UpsilonInc organization and another remote. These references do not prove the current total number of independent repositories.

Known code layout:

```
tapreco/
  apps/
    mutual-fund-web/
    estimated-tax-web/
    audit-risk-web/
    entity-comparison-web/
  services/tools-api/
  shared-ui/
  docs/
  scripts/
```

Repository count: At least one named Tapreco code repository is known. Four frontend folders and one backend folder are **applications/services inside a codebase**, not automatically five GitHub repositories. Exact repo count must be checked in the GitHub organization and each deployment service configuration.

Branching: Past discussions referenced main, develop, feature/tools-integration and tools_integration_relesaes. Confirm which branch each production service actually deploys.

8. Deployments: what happens when code is pushed

Frontend: Developer commits and pushes to a configured GitHub branch. Cloudflare Pages checks out the repository, runs the appropriate build command (such as npm run build), and serves the resulting Vite dist assets. Each frontend may be a separate Pages project even if all are in one repository.

Backend: Render checks out its configured repository/branch, installs dependencies, starts the Node.js service and exposes HTTPS API endpoints. The backend reads secret environment variables for database access.

Database: PostgreSQL is a managed service; code deployments do not automatically change schemas unless migrations are deliberately run. Previous discussions referenced npm run db:migrate in services/tools-api.

DNS: GoDaddy DNS records map desired domains/subdomains to Cloudflare or Render services. Exact Tapreco hostnames and DNS targets have not been independently verified.

9. Shared authentication and suite navigation

The desired experience is one login across Tapreco tools, with consistent headers, sign-out behavior, role checks and session refresh. Previously discussed roles include Admin, Firm Admin, CPA/Reviewer and Employee. The Tax Planner vendor integration needs a secure SSO or token-exchange design to avoid a second login. Exact implementation must be inspected before claiming full SSO.

When signing out, applications should clear their authenticated session consistently and avoid leaving stale access in another tool. Cross-subdomain cookie scope, token expiry, refresh behavior and redirect destinations should be tested.

10. Reports and export behavior

Known product requirements include PDF and Excel report output and one-click PDF review. The Audit Risk Analyzer generates risk findings and a PDF report; other tools have discussed report layouts. The exact list of report endpoints and templates needs code review. Do not confuse **report count** with **repository count**.

11. Security, operations and monitoring

Recommended controls: enforce HTTPS; keep database credentials server-side; validate all API inputs; implement role-based authorization; use a pooled PostgreSQL client; add request IDs and structured logs; restrict CORS; rotate secrets; monitor Render errors and response times; configure database backups; and test authentication across all apps.

12. Inventory requiring live confirmation

Item	Current understanding	How to verify
GitHub repository count	At least 1 named; exact total unknown	List repositories in organization and remotes
Cloudflare Pages projects	Portal plus custom frontends discussed	Cloudflare Pages dashboard
Render services	One shared tools-api discussed	Render service dashboard
Database instances	Supabase for tools; Neon for vendor	Connection configuration and provider dashboards
Exact domains / URLs	Not confirmed in this document	GoDaddy DNS and deployment custom domains
API routes	Shared Express API; route list unknown	Inspect services/tools-api routes
Live tool status	Four custom tools hosted; vendor separate	Open production URLs and test
Report templates / count	Not verified	Inspect PDF/Excel generation code

13. Practical example - Estimated Tax calculation

1. User opens the Estimated Tax frontend served by Cloudflare Pages.
2. React loads and displays the form.
3. User enters filing status, income and other inputs.
4. React sends the input to the Render API over HTTPS.
5. Express validates the request and performs calculation logic; it may query Supabase if stored data is required.
6. API returns results as JSON.
7. React displays tax estimates and offers any supported report/export options.

14. Next verification checklist

For a fully audited version, collect the GitHub organization repository list, Cloudflare Pages project list with production domains, Render service name and deployed branch, Supabase project identifier (not secrets), Neon ownership details, and API route inventory. Never include passwords, tokens, or private connection strings in the report.

Conclusion: The known architecture is a React/TypeScript multi-app frontend suite on Cloudflare Pages, a shared Node.js/Express API on Render, Supabase PostgreSQL for custom tools, and a separate vendor Tax Planner using Neon. One named Tapreco repository is known; the total repository count remains to be confirmed.